OpenSearch Korea User Group 발표 스크립트 — meta-repository와 memory-mcp

Oct 11, 2026 · @jin

발표 개요

여러 repository에 흩어진 개발문서를 meta-repository로 모으고, OpenSearch 기반 memory-mcp로 코딩 에이전트에게 먹여서 매 세션 컨텍스트를 다시 설명하는 비용을 없앤 이야기입니다. 분량은 발표 50분 + 질의응답 10분 기준으로 썼고, 섹션마다 괄호 안에 예상 시간을 적어 두었습니다.

항목	내용
청중	OpenSearch Korea User Group. OpenSearch 운영 경험이 있는 엔지니어
출발점	컨텍스트는 소유하는 것이다. history는 전화번호처럼, 다 외울 필요 없이 필요할 때 찾을 수 있어야 한다. 그 연락처를 팀 단위로 만든 네 단계의 이야기
핵심 메시지 1	OpenSearch는 처음이 아니라 3단계에 들어온다. 시작은 meta-repository 디렉토리 구조와 INDEX.md면 충분하다
핵심 메시지 2	쌓인 문서는 반드시 풍화한다. 60일 생명주기와 compaction이 없으면 토큰만 태운다
핵심 메시지 3	episodic 문서와 knowledge 문서는 저장소도 조회 방식도 다르다. episodic은 OpenSearch, knowledge는 graph DB
말투	구어체 존댓말. 대괄호 `[ ]`는 발표자 지시문이라 읽지 않습니다

1. 오프닝: 컨텍스트는 소유하는 것 (2분)

(슬라이드 1: 표지)

안녕하세요. 당근 광고실에서 데이터 플랫폼을 만드는 Jin입니다. 어트리뷰션 데이터를 모으고, 광고 서빙에 들어갈 데이터를 만듭니다. 스키마가 서비스에 묶여 있어서 repository를 예닐곱 개씩 동시에 열어 놓고 일합니다.

(슬라이드 2: "당신의 지능을 소유하라")

오늘 이야기는 한 문장에서 시작합니다. 와이콤비네이터 대표 개리 탠이 올해 8월 스타트업 스쿨 키노트에서 한 말입니다. "당신의 지능을 소유하라." 요지는 세 줄입니다. 모델은 빌리는 것이고 컨텍스트는 소유하는 것이다. 모델이 좋아질수록 차별점은 컨텍스트로 옮겨간다. 그리고 모델이 업그레이드되면 그 혜택은 내가 가진 컨텍스트 위에 공짜로 붙는다.

오늘은 그 컨텍스트를 개인이 아니라 팀이 어떻게 소유하고 있는지에 대한 이야기입니다.

2. 3개월 전에 입사했다면 (2분)

(슬라이드 3: 질문)

질문 하나 드리겠습니다. 여러분이 3개월 전에 입사했다고 해 보죠. 누가 와서 지난 1년 동안 팀이 내린 결정 하나를 짚으면서, 그때 왜 그렇게 했냐고 묻습니다. 뭐라고 하실 건가요?

[잠깐 멈춘다]

아마 "저는 그때 없었는데요"일 겁니다. 조금 더 성의 있게 답하면 "이유가 있었겠죠?" 정도겠죠. [웃음을 기다린다] 그래서 이유를 찾으러 가 보면, 혹시 그 팀의 문서는 슬랙 메시지인가요? 검색하면 스레드 몇 개가 나오고, 결정의 이유는 그 사이 어딘가에 흩어져 있습니다.

사람도 이런데, 매 세션 기억이 비어서 시작하는 코딩 에이전트는 어떨까요. 에이전트는 매 세션 오늘 입사한 사람입니다.

3. history는 전화번호처럼 (1분)

(슬라이드 4: 전화번호)

전화번호 몇 개나 외우고 계세요? 와이프 전화번호는 당연히 외우겠죠. [웃음] 그런데 우리 팀 리더 전화번호는 몰라도 됩니다. 010으로 시작하는 건 알죠. [웃음] 필요하면 연락처에서 찾으면 되니까요.

저는 history도 전화번호처럼 되어야 한다고 생각합니다. 매일 쓰는 몇 가지는 머리에 있고, 나머지는 다 외울 필요 없이 필요할 때 정확히 찾을 수 있으면 됩니다. 사람도 그렇고, 에이전트도 그렇습니다. 오늘은 그 연락처를 팀 단위로 만든 이야기를 하겠습니다.

4. 개발문서는 어떻게 관리하시나요 (3분)

(슬라이드 5: 질문 하나)

하나 여쭤보겠습니다. 코딩 에이전트를 쓰다 보면 개발문서나 스펙 문서가 계속 생기는데, 다들 그 문서를 어떻게 관리하고 계신가요?

[손을 들게 하면서 하나씩 묻는다]

- 혹시 meta-repository를 운영하시는 분 계신가요?
- 아니면 repository마다 작업할 때 스펙 문서를 만들어서 그 안에 넣어 두시나요?
- 아니면, AI가 만든 문서는 찌꺼기고 SSoT는 언제나 소스코드다, 이런 철학으로 가시는 분?

[잠깐 반응을 보고]

세 번째에 손 드신 분들, 저도 그 마음 압니다. 각자 철학이 있으실 텐데, 오늘은 제 이야기를 해 보려고 합니다.

(슬라이드 6: 매일 겪던 일)

저는 에이전트에게 일을 시킬 때마다 "이 프로젝트는 이런 배경이고, 이 용어는 이런 뜻이고, 지난번엔 이렇게 결정했고" 하며 컨텍스트를 길게 적어 주는 과정이 너무 반복적이고 피로했습니다. 그 피로를 없애려고 무엇을 했는지, 그리고 그 과정에서 OpenSearch가 어디에 들어갔는지를 말씀드리겠습니다.

5. 왜 repository 하나로는 안 되는가 (3분)

(슬라이드 7: 팀이 하는 일)

제가 속한 팀은 광고실 데이터 플랫폼팀입니다. 하는 일은 두 문장으로 요약됩니다. 광고 어트리뷰션을 수집해서 제공한다. 활성 광고 데이터를 만들어서 서빙 파트에 넘긴다. 이 중 두 번째, 활성 광고 데이터를 만드는 쪽에 비즈니스 로직이 많이 들어 있습니다.

이 데이터가 서비스의 어느 코드에서 어떻게 넘어왔는지를 알아야 제가 하는 작업의 사이드 이펙트까지 정확히 추론할 수 있습니다. 그런데 저희는 데이터 스키마가 아직 서비스 레이어와 강하게 결합된 플랫폼을 운영합니다. 그래서 repository 하나만 보면 되는 게 아니라 여러 개를 동시에 봐야 했습니다.

(슬라이드 8: 매일의 상황)

구체적으로 이런 상황입니다.

- 제가 한 줄도 기여하지 않은 repository의 코드를 반드시 읽어야 한다.
- 다른 팀 repository가 어느 방향으로 움직이는지도 알고 있어야 한다.
- 그 사실을 에이전트에게 매 세션 처음부터 다시 설명해야 한다.

세 번째가 진짜 문제였습니다. 설명을 빼먹으면 에이전트는 있지도 않은 history를 멋대로 추론해서 결과물을 만들고, 그걸 고치다 보면 하루가 가 있었습니다.

그래서 "모든 repository의 상태와 방향을 한곳에서 추적하는 repository를 만들자"고 생각했습니다. 그게 meta-repository의 시작입니다.

6. 얻고 싶었던 것, 그리고 네 단계 (3분)

(슬라이드 9: 얻고 싶었던 것)

이 meta-repository로 얻고 싶었던 건 사실 딱 하나였습니다.

AI가 알아서 그 repository의 history를 이해했으면 좋겠다.

예를 들어 팀에서 기획할 때 쓰던 용어와 실제 코드가 정확히 연결됐으면 좋겠다는 겁니다. 기획서에는 "활성 광고"라고 쓰여 있는데 코드에는 다른 이름으로 돼 있고, 그 사이의 변천사는 저만 아는 상황. 다들 한 번쯤 겪어 보셨을 겁니다. 정리하면 요구는 두 줄입니다.

- 매 세션 지난 history를 설명하지 않아도, 없는 history를 지어내지 말 것.
- 특정 repository 작업이 시작되면 알아서 지난 history를 읽어 갈 것.

(슬라이드 10: 네 단계)

먼저 말씀드릴 것이 있습니다. 처음부터 memory-mcp를 만든 게 아닙니다. 지금까지 네 단계를 거쳤고, 각 단계는 앞 단계가 남긴 문제에서 나왔습니다.

1. 매 세션 history를 다시 설명해야 해서, meta-repository와 디렉토리 구조를 만들었습니다. 여기까지는 OpenSearch 없이 INDEX.md만으로 운영했습니다.
2. 문서가 밖에도 있었고 쌓인 문서는 낡아서, 바깥 문서를 불러오고 문서에 생명주기를 줬습니다.
3. meta-repository가 너무 커져서 탐색이 오래 걸리자, 그제서야 OpenSearch 기반 memory-mcp를 붙였습니다.
4. 사실과 유효한 규칙을 늘 최신으로 유지해야 해서, knowledge를 graph DB로 옮겼습니다.

하나씩 보겠습니다.

7. 1단계: meta-repository와 디렉토리 구조 (6분)

(슬라이드 11: meta-repository)

meta-repository라는 말을 들어 보셨나요? 간단합니다. 여러 repository의 메타 정보를 한곳에 모아 둔 repository입니다.

여기서 "메타 정보"가 무엇이냐가 중요한데, 저는 네 가지로 정했습니다. 그 repository의 spec, ADR, PRD, 그리고 AI가 작업 전에 세운 plan 문서입니다. 이걸 meta-repository에 저장하게 하고, 원래 repository 안에 docs 디렉토리가 이미 있다면 거기서 정보를 끌어다 쓰도록 가이드를 적어 두었습니다.

(슬라이드 12: 디렉토리 구조)

실제로 관리하는 meta-repository는 이렇게 생겼습니다.

```
meta-repo/
├── CLAUDE.md
├── repos/                              # 관리 대상 repository (함께 체크아웃)
└── docs/
    ├── repository/{repo}/
    │   └── INDEX.md
    ├── feature/{yyyy-mm-dd}_{topic}/
    │   └── INDEX.md
    ├── specs/{yyyy-mm-dd}_{spec}/
    │   └── INDEX.md
    └── knowledge/{topic}/
        └── INDEX.md
```

repository 아래에는 그 repository의 ADR, 배경, 용어 매핑을 둡니다. feature는 여러 repository에 걸친 기능, specs는 사람이 쓴 스펙과 AI가 만든 스펙, 작업 plan입니다. knowledge에는 주제별 사실과 규칙만 둡니다.

모든 디렉토리에 INDEX.md를 둡니다. 사람에게는 진입점이고, 에이전트에게는 전체를 읽기 전에 먼저 읽는 목차입니다. 이 INDEX.md 덕분에 초기에는 OpenSearch를 붙이지 않고도 운영할 수 있었습니다. 나중에 검색엔진이 들어오면서 INDEX.md를 운영할 필요는 사라졌습니다.

(슬라이드 13: 디렉토리마다 전제가 다르다)

이 구조에서 중요한 건 디렉토리마다 깔린 전제입니다.

feature와 specs의 하위 디렉토리는 전부 {yyyy-mm-dd}_{주제}로 만들어집니다. 그 안의 데이터는 거의 전부 episodic입니다. 그날 무슨 일이 있었고 무엇을 정했는지의 기록이라, 날짜를 탑니다.

반대로 repository 아래의 모든 문서는 언제나 최신이라는 전제를 깝니다. 그 repository의 지금 모습입니다.

knowledge는 하위 디렉토리가 그냥 주제입니다. 사실과 규칙만 적혀 있고, 그래서 날짜도, 생명주기도 타지 않습니다. 이 구분이 뒤에 나오는 2단계 생명주기와 4단계 knowledge graph의 바탕이 됩니다.

(슬라이드 14: worktree와 CLAUDE.md)

관리 대상 repository는 meta-repository 안에 함께 체크아웃됩니다. 작업은 항상 새 worktree를 만들어서 합니다. 이유는 세 가지입니다.

- 에이전트가 여러 작업을 동시에 하면 브랜치가 섞이는데, worktree면 디렉토리가 분리됩니다.
- 작업 경로가 고정돼 있어서 어떤 변경이 어느 작업에서 왔는지 추적됩니다.
- 뒤에서 말씀드릴 codebase-memory-mcp가 worktree를 감지해서 코드 위치를 답해 줍니다.

그리고 CLAUDE.md를 만들었습니다. 이 repository를 에이전트가 어떻게 써야 하는지가 적혀 있습니다. 작업을 시작할 때 그 repository의 INDEX.md부터 읽을 것, 3단계부터는 search_memory를 먼저 부를 것. 결과 문서는 어느 디렉토리에 어떤 frontmatter로 저장할 것. 그리고 이 규칙들의 변경 내역입니다. 변경 내역이 있어야 에이전트가 "예전 규칙으로 만들어진 문서"를 구분할 수 있습니다.

하나 더 있습니다. 문서를 보고 정보를 만들 때는 지시문에 "추론 없이"라는 말을 반드시 넣었습니다. 이 한마디의 효과가 생각보다 컸습니다.

(슬라이드 15: 스킬)

그리고 스킬을 만들었습니다. Jira나 Linear 티켓만 입력하면 작업을 하고, 문서를 만들고, 티켓 상태를 옮기고, Slack으로 알림을 주고, 블랙박스 테스트를 설계해서 Linear에 댓글로 달고, 배포하고, e2e 테스트까지 자동으로 돌아갑니다.

거창해 보이지만 대강 만들었습니다. 대충, AI한테 만들어 달라고 해서요. [웃음] 중요한 건 세 번째입니다. 작업이 끝나면 문서가 meta-repository에 남습니다. 문서를 쓰는 일이 사람의 숙제가 아니라 작업의 부산물이 됩니다.

그런데 문서가 쌓이기 시작하자 두 가지가 보였습니다. 문서는 meta-repository 밖에도 있었고, 쌓인 문서는 시간이 지나면 낡았습니다.

8. 2단계: 문서를 불러오고, 생명주기를 준다 (3분)

(슬라이드 16: 문서를 불러온다)

문서가 생기는 길은 세 가지입니다. 기능을 추가로 개발하거나 수정할 때, Notion 문서를 싱크할 때, 그리고 관리하는 repository에 있는 모든 문서입니다. 이렇게 들어온 문서는 전부 feature로 가서 episodic으로 쌓입니다. knowledge에는 바로 들어가지 않습니다. 바로 다음에 말씀드릴 생명주기에서 살아남은 사실과 규칙만 knowledge로 갑니다.

(슬라이드 17: 문서의 생명주기)

repository의 방향과 정보는 시간이 흐르면 풍화됩니다. 더 이상 유효하지 않은 문서가 그대로 남아 있으니, 에이전트가 쓸데없이 많은 문서를 탐색하며 토큰만 태웠습니다.

그래서 문서에 생명주기를 줬습니다. 대상은 언제나 60일이 지난 episodic 문서, 그러니까 feature와 specs입니다. 60일은 문서 나이로 순니다.

1. 60일이 지난 문서를 대상으로 새 문서를 만들어, compaction된 결과를 저장합니다.
2. 이 결과는 feature나 knowledge에 새 문서로 추가되거나, 기존 문서에 append됩니다. 이때 원문의 S3 경로를 함께 남깁니다.
3. 60일이 지난 원문은 remote storage인 S3로 옮깁니다.
4. 이 수정이 PR로 만들어집니다.

지우지 않는다는 게 핵심입니다. 원문은 cold로, 살아남은 내용은 hot으로 갑니다. 그리고 knowledge는 생명주기를 타지 않습니다. 한 번 들어간 사실과 규칙은 날짜가 지나도 cold로 가지 않습니다.

[OpenSearch를 쓰는 분들께는 단어가 다르게 들릴 수 있어 한 번 짚는다] 여기서 compaction은 세그먼트 병합이 아니라 문서를 다시 써서 줄이는 일이고, cold는 데이터 티어가 아니라 S3에 보관하는 것입니다. hot/cold라는 개념이 여기서 처음 생겼지만, 이때의 hot/cold는 아직 OpenSearch가 아니라 파일과 S3 경로였습니다.

그런데 2단계까지 오니 다른 문제가 생겼습니다. meta-repository에 들어 있는 문서가 너무 커졌습니다.

9. 3단계: OpenSearch memory-mcp (9분)

(슬라이드 18: 3단계로 넘어간 이유)

같은 단어, 비슷한 문장을 쓰는 문서가 여러 개 있으면 에이전트는 그걸 전부 읽어 봅니다. 파일 트리를 탐색하고, ag로 검색하고, 열고, 읽고, 비교합니다. 비슷한 문서마다 이 일이 반복되니 시간과 토큰이 쌓입니다.

이 구간은 검색엔진이 수십 년 동안 해 온 정보 검색, IR 그 자체입니다. 그래서 색인하고 찾는 일을 에이전트가 직접 하는 게 아니라, OpenSearch가 IR을 맡고 에이전트는 결과만 받게 하자. 이것이 3단계입니다.

(슬라이드 19: 요구사항)

제가 필요한 건 분명했습니다. 시간순으로 계속 쌓이는 episodic 문서에서 지식을 꺼내 오는 것. 완벽한 하나의 knowledge base를 만드는 건 이 단계에서 포기했습니다. 하나의 리소스를 두고 "A냐 B냐", "A와 B를 합쳐 C냐" 같은 충돌 관리는 감당하기엔 너무 무겁습니다. 조건은 네 가지였습니다.

번호	요구사항	이유
1	GitHub나 로컬에 있는 문서 파일을 OpenSearch에 담을 것	문서는 이미 meta-repository에 파일로 있음
2	팀/실 단위로 문서를 분리 관리할 것	광고실 안에서도 팀마다 문맥이 다름
3	hot/cold 영역이 있을 것	1년, 2년 지난 문맥을 참조할 일이 생길 수 있음. 오래된 history도 오래 남기고 싶었다
4	cold는 S3에 적재하고, 필요하면 OpenSearch로 바로 로드할 것	지우긴 아깝고, 항상 켜 두긴 비쌈

[청중에게] 왜 SQLite나 PostgreSQL이 아니냐고 물으실 수 있는데, 생각을 안 해 본 건 아닙니다. 다만 OpenSearch를 다룰 줄 아는 사람이 굳이 다른 걸 고를 이유가 없었습니다. 여기 계신 분들도 아마 같은 결론이실 겁니다.

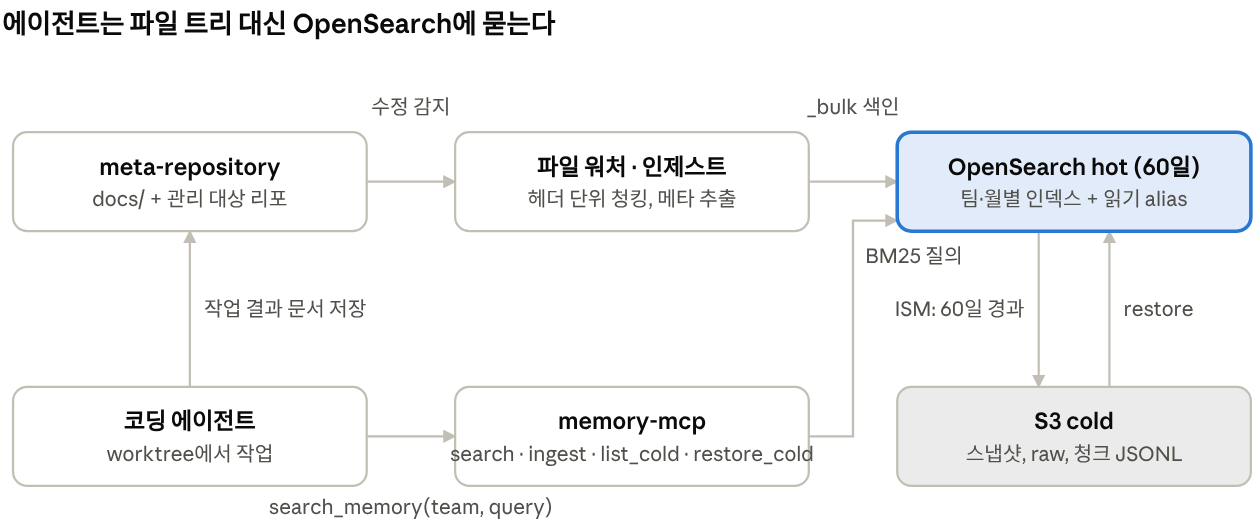
[슬라이드에는 없는 내용. 말로만 짧게] 먼저 남이 만들어 둔 걸 찾아봤습니다. OpenSearch 공식 MCP 서버의 memory 도구도, 커뮤니티 memory MCP도, SQLite 기반 로컬 마크다운 RAG MCP들도 색인과 질의는 잘 되어 있었습니다. 그래도 공식 서버를 굳이 쓰지 않고 도구 네 개를 직접 만들었습니다. 이유는 솔직하게 이렇습니다. 만들면 되니까요. 팀과 조직 개념을 넣고 싶었고, 공식 서버를 잘 몰랐고, 배우려는 목적도 있었습니다. 만들어 보니 도구 자체는 생각보다 작았습니다.

(슬라이드 20: 기능 여섯 가지)

1. OpenSearch는 우선 로컬 머신에 직접 올립니다. 팀 공용 클러스터는 나중 문제입니다.
2. 일단 전부 넣습니다. meta-repository의 docs와 관리 대상 repository의 docs가 모두 색인 대상입니다.
3. 파일 워칭을 합니다. docs 아래에서 발생하는 모든 수정은 수정 색인 대상입니다.
4. 처음에는 임베딩을 쓰지 않고 BM25만 씁니다. 비용 문제입니다. 추후 remote 모드로 올릴 때 임베딩을 붙입니다.
5. hot만 OpenSearch에 적재합니다. 기준은 2단계에서 정한 60일 그대로입니다.
6. cold가 필요하면 S3에서 내려받아 OpenSearch에 다시 적재합니다.

2번이 제 솔직한 마음입니다. 보통은 다 때려 넣습니다. 아무리 못 만들어도 ag로 검색하는 것보다는 낫겠다는 생각이었습니다. [웃음]

(슬라이드 21: 구성)



memory-mcp 구성 · 상단은 문서가 들어가는 길, 하단은 에이전트가 묻는 길

위쪽 줄은 문서가 색인되는 길이고 아래쪽 줄은 에이전트가 묻는 길입니다. 60일이 지난 인덱스는 ISM이 S3로 보내고, 필요할 때만 restore로 돌아옵니다.

BM25만으로 되느냐. 됩니다. 검색 주체가 사람이 아니라 에이전트라서, 에이전트가 던지는 질의 자체가 자연어 문장이고 BM25는 그 토큰으로 점수를 매깁니다. 특히 개발문서는 함수명, 서비스명, 에러 코드, 파일 경로 같은 식별자가 강한 어휘 신호라서 BM25가 잘 맞는 경우가 많습니다. 어휘가 안 맞으면 에이전트가 질의를 두세 번 바꿔 다시 부르면 되고, 그건 MCP 도구 호출 루프에서 자연스럽게 일어납니다.

(슬라이드 22: hot/cold 구성)

hot/cold는 어떻게 구현했느냐. episodic 문서는 추가만 되고 시간순으로 쌓이니 로그 데이터와 성질이 같습니다. 그래서 로그 운영 패턴을 그대로 가져왔습니다.

레이어	구성
인덱스	`mem-{team}-{yyyy.MM}` 월별 인덱스. 읽기 alias가 hot 인덱스 전체를 가리킴. 요구사항 2번(팀 단위 분리)은 인덱스 이름과 alias로 끝남
문서 필드	team, repo, doc_type(spec/ADR/PRD/plan), event_time, source_path, git_sha, header_path, 본문 text
청킹	마크다운 헤더 계층 단위. 코드 블록 안에서는 안 자름
ISM 정책	hot 60일 → 스냅샷으로 S3 repository에 저장 → 인덱스 삭제. 필요하면 스냅샷 restore로 인덱스 단위 재로드
S3 층	원본 raw, 파싱된 청크 JSONL, 스냅샷. 매핑을 바꾸거나 나중에 임베딩을 붙일 때 JSONL에서 _bulk로 재적재

분석기는 nori입니다. 인덱스 기본 분석기를 nori로 두었습니다. 한국어 문서와 기획 용어가 섞여 있으니까요. 그리고 knowledge 문서는 ISM 대상이 아닙니다. 생명주기를 타지 않으니까요.

(슬라이드 23: MCP 도구)

MCP 도구는 네 개면 충분했습니다.

도구	역할
`search_memory(team, query, time_range)`	hot alias에 BM25 검색. 결과는 source_path와 header_path를 포함한 청크
`ingest_path(path, team)`	워처가 놓친 경우를 위한 수동 적재

도구	역할
`list_cold(team)`	S3에 있는 cold 인덱스 목록
`restore_cold(index)`	스냅샷 restore. 에이전트가 cold 범위 질의를 받으면 "cold 인덱스 X가 있는데 복원할까요"를 먼저 물음

10. 데모와 측정 (6분)

(슬라이드 24: 데모)

[데모 화면 전환]

짜잔. 지금은 docs의 문서를 OpenSearch에 넣어 두고 검색하는 memory-mcp가 돌아가고 있고, 에이전트는 작업할 때마다 이 문서를 참고하며 일합니다. 앞에서 말씀드린 "활성 광고" 예시를 그대로 써 보겠습니다.

[데모 순서]

1. meta-repository의 docs 아래에 "활성 광고" 용어 매핑을 담은 ADR 하나를 저장한다. 워처가 잡아서 색인되는 로그를 보여 준다.
2. 에이전트 세션을 새로 열고, "활성 광고"와 관련된 작업을 시킨다. 배경 설명 없이.
3. 에이전트가 `search_memory`를 부르고, 돌아온 청크의 `source_path`를 근거로 작업하는 걸 보여 준다.
4. 같은 작업을 memory-mcp 없이 돌린 화면과 나란히 본다. 따로 잰 수치는 없으니 화면의 차이로 보여 준다.

(슬라이드 25: 측정)

솔직하게 말씀드리면 숫자는 없습니다. 따로 재 보지는 않았습니다. 그런데 써 보면 압니다. [잠깐 멈춘다] 에이전트가 파일 트리를 뒤지고 같은 문서를 몇 번씩 열던 일이 줄었고, 그만큼 시간과 토큰이 눈에 띄게 줄었습니다.

도구 자체는 정말 쉽게 만들어졌습니다. 그런데 episodic 문서는 이렇게 다룰 수 있어도, knowledge는 달랐습니다.

11. 4단계: knowledge graph DB (3분)

(슬라이드 26: 4단계로 넘어간 이유)

지금 memory-mcp에 들어가는 문서는 거의 전부 episodic입니다. "그날 그 세션에서 무슨 일이 있었나"를 적은 기록이라, 시간이 붙어 있고 날짜가 지나면 가치가 떨어집니다. 그래서 60일이 지나면 cold로 보내도 아쉽지 않습니다. 반대로 knowledge는 "이 팀은 이렇게 한다"처럼 시간이 붙지 않는 사실과 규칙입니다. 한 번 맞으면 오래 맞고, 충돌이 나면 어느 쪽이 맞는지 따져야 합니다.

구분	episodic 문서	knowledge 문서
정체	시간순으로 쌓이는 작업 기록. ADR, plan, 회의 결정	걸러져 살아남은 사실. 용어 정의, 시스템 불변식
저장소와 IR	역색인 DB. 지금은 OpenSearch BM25	graph DB(neo4j). 파일이 아닌 원격 저장소
생명주기	반드시 있어야 함. 60일 hot, 이후 cold	영속. 대신 갱신 이력이 필요
사실 충돌	무관해도 됨. 같은 주제에 다른 기록이 공존하는 게 정상	항상 주의. A/B 중 택일하거나 C를 만들어야 함
흐름	compaction 때 살아남은 사실이 knowledge로 넘어감	episodic에서만 들어옴

핵심은 충돌에 대한 태도입니다. episodic은 충돌을 신경 쓰지 않아도 되니 설계가 가볍고, 그래서 먼저 만들 수 있었습니다. knowledge는 항상 신경 써야 해서 따로 운영해야 했습니다. 그래서 knowledge는 생명주기를 타지 않습니다.

(슬라이드 27: knowledge graph DB)

그래서 4단계에서는 knowledge를 graph DB, neo4j로 옮겼습니다. 목적은 하나입니다. 사실과 유효한 규칙을 늘 최신으로 유지하는 것. 들어오는 길은 2단계 compaction에서 살아남은 사실과 규칙입니다.

해 보니 의외였던 점이 하나 있습니다. 생각보다 이 정보는 파일보다 원격 저장소가 더 좋았습니다. [왜 더 좋았는지 한두 문장, 발표 전에 채울 것]

(슬라이드 28: 데모 저장소)

[저장소를 열어 지금까지의 구조를 보여 준다. 무엇을 보여 줄지는 발표 전에 채울 것]

지금까지 말씀드린 구조는 이 저장소에 있습니다. github.com/drakejin/workspace입니다.

12. 다른 곳의 사례: 컨텍스트 엔지니어링 (4분)

(슬라이드 29: 같은 문제, Elastic의 컨텍스트 엔진)

여기까지가 제가 해 온 일입니다. 그런데 이 문제를 저만 풀고 있는 건 아니었습니다. 올해 10월 8일, Elastic 공식 블로그 Search Labs에 "Context engineering for agents"라는 글이 올라왔습니다.

글의 문제의식은 이렇습니다. 지금까지의 컨텍스트 엔지니어링은 에이전트 하나의 컨텍스트 창에 무엇을 넣을지에 집중했다. 더 어려운 문제는 대화가 시작되기 전에 컨텍스트를 만들어 두고, 여러 에이전트가 함께 쓰게 하는 것이다. 한 문장으로는 이렇습니다. "컨텍스트 엔진은 질문이 오기 전에 데이터를 이해한다."

이 글이 꼽은 문제 세 가지는 저희가 겪은 것과 같았습니다. 질의마다 드는 탐색 비용은 1단계 INDEX.md와 3단계 OpenSearch로 대응했고, 컨텍스트 표류, 즉 손으로 쓴 문서가 낡는 문제는 2단계 생명주기로 대응했습니다. 고립된 지식, 한 에이전트가 배운 것이 다른 에이전트를 돕지 못하는 문제는 아직입니다. 저희 OpenSearch는 아직 로컬 머신에 있습니다.

Elastic은 수치도 내놓았습니다. Elastic은 같은 질문, 같은 모델로 도구 호출이 33번에서 9번, 입력 토큰이 71% 줄고, 응답이 213초에서 64초가 됐습니다. 저희는 앞에서 말씀드린 대로 재 보지 않았습니다. 비슷하겠죠? [웃음] 데이터 영역도 다르니 방향만 보시면 됩니다.

(슬라이드 30: 루프에 대입하면)

Elastic은 이 컨텍스트 레이어를 원본 데이터와 에이전트 사이에 두고, 네 단계가 끊기지 않고 도는 루프로 설명합니다. 수집, 구축, 검색, 그리고 개선. 개선 단계에서 에이전트의 사용 기록이 새 소스로 다시 들어가야 루프가 됩니다. 그리고 네 단계 모두 검색 문제라고 봅니다. "에이전트의 컨텍스트 레이어를 설계한다면, 검색 위에 지어라." 이 자리에 계신 분들께는 반가운 결론이죠. [웃음]

오늘 말씀드린 네 단계를 이 루프에 대 보면, 수집과 구축은 대응했습니다. 소스는 전부 feature로 들어오고, compaction이 사실과 규칙을 knowledge로 옮깁니다. 다만 Slack 대화나 티켓 댓글은 소스가 아니고, 지식 단위도 사실과 규칙 두 가지뿐입니다. 검색은 부분 대응입니다. BM25에 nori까지는 했지만 의미 검색이 없고, 도구 네 개가 모두 OpenSearch를 향해 있어 knowledge graph까지 한 번에 묻지 못합니다. 그리고 개선은 비어 있습니다. 규칙을 바꾸는 것도, compaction PR을 검수하는 것도 사람이 합니다. 에이전트가 무엇을 못 찾았는지, search_memory를 아예 안 불렀는지는 다시 들어오지 않습니다.

(슬라이드 31: 빠진 고리)

정리하면 지금 구조는 루프가 아니라 한 방향 파이프라인입니다. 쓰임이 다시 들어와야 루프가 됩니다. 메울 방향은 네 가지로 보고 있습니다.

1. **사용 기록을 소스로.** search_memory의 질의와 결과, 도구 호출을 남기고 compaction 때 함께 봅니다. 무엇을 못 찾았는지가 남아야 다음 compaction이 무엇을 만들지가 바뀝니다.
2. **검색 품질을 잰다.** 질의 세트와 판정 목록으로 BM25 결과를 재고, 바꿀 때마다 실험으로 비교합니다.
3. **팀이 함께 쓰는 클러스터.** 로컬 OpenSearch를 팀 공용으로 올려서 한 사람의 에이전트가 배운 것을 팀이 쓰게 합니다.
4. **한 번에 묻기.** episodic과 knowledge를 한 질의로, BM25에 의미 검색을 더해서.

반가운 건 이걸 새로 만들 필요가 없다는 겁니다. OpenSearch에는 사용자 행동 기록을 남기는 UBI와, 질의 세트·판정 목록·실험을 다루는 검색 적합성 워크벤치가 이미 있습니다. 공식 OpenSearch MCP 서버에도 도구로 들어 있습니다. Elastic 글이 말한 개선 단계를, 저희는 OpenSearch로 채울 수 있습니다.

[메울 방향의 우선순위는 발표 전에 정리할 것]

13. 함께 쓰면 좋은 조합 (2분)

(슬라이드 32: 마무리)

[구분 장표. 같이 쓰면 좋았던 조합, 말하고 싶은 것, 열린 질문 순서로 닫는다]

(슬라이드 33: 같이 쓰면 좋았던 세 가지)

memory-mcp 하나로 모든 게 해결되지는 않습니다. 같이 써 보니 좋았던 세 가지를 말씀드리겠습니다.

첫째, codebase-memory-mcp. 문서가 아니라 코드의 위치를 답해 주는 도구입니다. 코드 구조를 그래프로 들고 있어서 "이 필드를 만드는 함수가 어디냐"에 파일 탐색 없이 답합니다. worktree도 감지해서 잘 답해 주는데, worktree에서 코드 변경을 어떻게 감지하는지 내부는 저도 아직 모릅니다. [솔직하게 말할 것] 문서는 OpenSearch, 코드는 codebase-memory-mcp로 나누면 에이전트가 파일 트리를 스캔하는 토큰이 크게 줄어듭니다.

둘째, 주석 기반 코드 history. 주석에 잡동사니를 넣는 것 같아 꺼려질 수 있습니다. 하지만 클래스 멤버 변수에 괴이한 history가 섞여 있다면, 왜 그런지를 주석으로 명시해 두는 편이 에이전트가 읽는 컨텍스트를 훨씬 명료하게 만듭니다. 문서는 코드와 떨어져 있지만 주석은 코드 옆에 붙어 있어서, 에이전트가 그 코드를 읽는 순간 함께 읽힙니다.

셋째, episodic과 knowledge의 분리. 앞에서 본 구분입니다. 처음부터 나눠 두면 hot/cold 기준도, 충돌 처리도 훨씬 단순해집니다.

14. 정리와 열린 질문 (3분)

(슬라이드 34: 다섯 가지)

하나. meta-repository부터 운영하세요. 디렉토리 구조와 INDEX.md만으로도 시작할 수 있습니다. repository마다 흩어진 문서는 에이전트가 찾지 못합니다. 찾지 못하면 지어냅니다.

둘. 문서에 생명주기를 주세요. 그래야 문서가 찌꺼기가 되지 않습니다. 처음에 "문서는 찌꺼기"에 손 드신 분들께 드리는 제 답입니다. 계속 가공하고, 압축하고, 최신화하세요. frontmatter, header_path, INDEX.md 같은 작은 구조가 검색 품질을 결정합니다. 그리고 문서가 커지면 IR은 에이전트가 아니라 OpenSearch에 맡기세요. 여기 계신 분들은 이미 그 도구를 운영하고 계십니다.

셋. 관리 대상 repository의 코드베이스에는 codebase-memory-mcp를 로컬에서 돌리세요. 매 세션 파일 탐색과 스캔에 쓰이는 토큰과 시간을 크게 아낄 수 있습니다.

넷. 메모리 설계가 에이전트의 성능을 가릅니다. 같은 모델, 같은 프롬프트라도 어떤 맥락이 앞에 붙느냐에 따라 결과가 완전히 달라집니다. 이 주장은 저 혼자의 경험이 아니라 업계에서 반복해서 나오는 이야기입니다. ["인용 문장" — 누구의 인용. 출처: 형식으로 두 건 채울 것]

다섯. knowledge는 graph DB로 운영하세요. 사실과 규칙은 파일보다 원격 저장소가 더 잘 맞았습니다.

(슬라이드 35: 열린 질문)

knowledge를 운영하면 바로 부딪히는 문제가 하나 있습니다. episodic에서 넘어온 사실이 기존 사실과 부딪히면 A인지 B인지 C인지를 누가 정하느냐입니다. 지금 생각하는 답은 단순합니다. 충돌하면 최신이 이기고, 최신이 거짓 같아 보이면 사람이 검수하는 프로세스를 두려고 합니다. 그런데 "거짓 같아 보인다"를 누가, 무엇으로 판단하는지는 아직 비어 있습니다.

(슬라이드 36: 감사합니다)

그래서 이 질문을 열어 둔 채로 마치려고 합니다. 여러분은 어떻게 하실 건가요. 정답을 드리러 온 것이 아니라 같이 이야기하러 왔습니다. 감사합니다. 질문과 이야기를 받겠습니다.

(슬라이드 37: 예상 질의응답과 참고 자료)

[질의응답 동안 띄워 둔다. 위쪽 여섯 칸은 15장의 예상 질문 중 가장 아프게 들어올 것들과 짧은 답이다. 답은 장표에 적힌 만큼만 말하고 청중 질문으로 넘긴다. compaction 검증과 문서 충돌은 아직 못 푼 문제라고 그대로 인정한다. 아래 세 줄은 인용한 영상, 블로그 글, 공식 MCP 서버의 링크]

15. 예상 질문과 답변

장표 37에는 이 중 여섯 개를 짧은 답과 함께 띄워 둔다. 공식 MCP 서버를 안 쓴 이유, ripgrep으로 충분하지 않나, knowledge는 왜 neo4j인가, compaction 검증, 문서 충돌, 에이전트가 search_memory를 안 부르면.

예상 질문	답변 요지
왜 OpenSearch 공식 MCP 서버의 memory 도구를 안 썼나	두 가지가 있는데, Memory Tools는 Amazon OpenSearch Service 2.19+ 또는 Serverless가 필요해서 로컬에서 못 씀. Agentic Memory는 3.3+ 자체 호스팅으로 되지만 평문 사실 저장용이라 파일 적재와 청킹이 없음. 공식 서버는 쓰지 않고 도구 네 개를 직접 만듦. 만들면 되니까. 팀·조직 개념을 넣고 싶었고, 잘 몰랐고, 배우려는 목적도 있었음. 공식 서버에도 SearchMemoryTool이 있어 이름이 겹치니 구분해서 말할 것
임베딩 없이 BM25만으로 검색 품질이 되나	개발문서는 식별자가 강한 어휘 신호라 BM25가 잘 맞음. 못 찾는 사례가 쌓이면 `knn_vector` 필드를 추가해 hybrid 쿼리로 올릴 계획. S3의 parsed JSONL에서 재적재하면 전체 재인덱싱이 쉬움
한국어 문서는 어떻게 하나	인덱스 기본 분석기를 nori로 둠. 한국어 문서와 기획 용어를 형태소 단위로 색인
hot/cold 경계를 왜 60일로 잡았나	오래된 문서를 손으로 지울 때 쓰던 기준을 그대로 가져옴. 팀마다 다를 수 있어서 ISM 정책의 숫자 하나로 바꿀 수 있게 해 둠
cold를 searchable snapshot으로 바로 검색하면 안 되나	가능하고 다음 옵션. 지금은 cold 질의가 드물어서 restore 방식으로 시작. 복원 전에 에이전트가 사용자에게 먼저 묻게 한 게 핵심
팀/실 단위 분리는 보안도 되나	지금은 인덱스 이름과 alias로 논리 분리만. 권한 분리가 필요하면 Security 플러그인의 인덱스 패턴 롤 매핑으로 확장
문서 충돌(collision)은 어떻게 처리하나	episodic 단계에서는 처리하지 않음. 같은 주제에 다른 기록이 공존하는 게 정상이고 시간 필터로 최신을 우선. knowledge로 넘길 때만 사람이 compaction에서 판단. 자동화는 미해결
에이전트가 search_memory를 안 부르면	CLAUDE.md에 작업 시작 규칙으로 명시. 그래도 빠지면 세션 시작 훅에서 repository 이름으로 한 번 자동 검색해 컨텍스트에 넣는 방식을 실험 중
토큰 절감 수치는	따로 재지 않았음. 써 보며 파일 탐색과 같은 문서 반복 읽기가 눈에 띄게 줄었다는 체감만 있다고 솔직하게 말할 것

예상 질문	답변 요지
코드를 공개하나	공개. 저장소는 github.com/drakejin/workspace (데모 장표와 마지막 장표에 주소가 있음)
knowledge는 왜 OpenSearch가 아닌가. 무슨 graph DB인가	neo4j. OpenSearch는 역색인 저장 구조로만 쓰고 그 이상의 역할을 주지 않는 게 마음이 편했음. [원격 저장소가 파일보다 나았던 이유는 발표 전에 채울 것]
Notion 싱크는 어떻게 하나	[싱크 방식과 주기를 발표 전에 채울 것]
Elastic 컨텍스트 엔진과 무엇이 다른가	Elastic은 데이터(색인, 계약서)에서 지식 단위를 미리 만들고, 우리는 개발문서에서 만든다. 가장 큰 차이는 개선 단계. Elastic은 에이전트 기록을 다시 소스로 넣고, 우리는 아직 못 했다
60일은 문서 나이인가, 인덱스 나이인가	문서 나이 기준. [ISM의 나이 조건은 인덱스 생성 시점 기준이라, 월별 인덱스와 어떻게 맞추는지 이어서 물을 수 있음]
그 규모면 ripgrep으로 충분하지 않나. BM25 품질 근거는	처음엔 ripgrep으로 했는데, 여러 번 문서를 찾아보고 탐색하는 게 더 토큰 낭비라고 판단함. 같은 단어가 들어간 문서가 여러 개일 때 그 뉘앙스대로 찾아 주는 건 OpenSearch가 더 잘할 거라는 직감으로 선택함
compaction 요약의 정보 손실이나 지어낸 내용은 누가 검증하나. PR 검수량은	검증하지 못했음. 이번 데모 예제의 맹점이라고 솔직하게 말할 것

16. 참고 출처

- Garry Tan: Own Your Intelligence — Y Combinator, Startup School 2026 (영상) — 1장(오프닝) "당신의 지능을 소유하라"의 원본 영상. 장표 37(예상 질의응답과 참고 자료)에 링크.
- Garry Tan: Personal AGI Is How You Stay Under Your Own Power — Startup School 2026, 2026-08-06 — 위 영상의 요약 글. 1장(오프닝)의 인용. "모델은 빌리고 컨텍스트는 소유한다"의 원출처.
- Context engineering for agents: 33 tool calls down to 9 — Elastic Search Labs, Anish Mathur, 2026-10-08 — 12장(다른 곳의 사례). 컨텍스트 엔진 정의, 세 가지 문제, 수집·구축·검색·개선 루프, 비교 수치(도구 호출 33→9, 입력 토큰 71% 감소, 213초→64초)의 출처.
- opensearch-project/opensearch-mcp-server-py — GitHub — 12장 빠진 고리. 검색 적합성 워크벤치와 UBI 도구, 공식 서버 Memory Tools 요건의 출처.

외부 인용은 개리 탠, Elastic 글, OpenSearch 공식 MCP 서버 세 곳이다. 나머지는 전부 발표자 본인의 이야기다. 13장 넷째의 인용 두 건은 발표 전에 출처를 확인해 여기에 추가할 것.